



Eigenface Recognition

Second interim report

Presented by: Kateryna Suchok
Anastasiya Shopska
Elizaveta Buhir
Oleksandr Tsybrivskiy

General topic and detailed description of the problem

The general topic of this project is the application of linear algebra, specifically the Singular Value Decomposition (SVD) and Principal Component Analysis (PCA), to automatic face recognition. The eigenface method, which our implementation follows, is built on a key insight: although a grayscale face image of dimensions $h \times w$ lives in a high-dimensional space $\mathbb{R}^d$ with $d = h \cdot w$, natural face images actually lie on a much lower-dimensional linear subspace. Our task is to find this subspace, represent faces compactly within it, and perform recognition using only a small set of coordinates.

The input to our system is a training set of labeled, aligned, grayscale face images of several subjects, together with a single query face image of unknown identity. The expected output is twofold. First, if the query sufficiently resembles the learned face subspace, the system returns a predicted identity label. Second, if the query does not conform to the subspace, meaning it is a non-face image or a face from a person not in the training data, the system rejects it and labels it as *unknown*. This rejection capability is essential for any practical recognition system.

To evaluate our eigenface implementation, we compare it against the Fisherfaces method, which is based on Linear Discriminant Analysis (LDA). While Eigenfaces finds directions of maximum variance in an unsupervised manner, Fisherfaces explicitly seeks directions that maximize separation between different identities while keeping images of the same identity close together. This comparison reveals the trade-off between generality and discriminative power. We compare both methods on identical datasets using metrics such as classification accuracy, robustness to small training sets, and the ability to reject unknown faces.

These technologies are widely used in real-world applications. Linear-algebra-based face recognition appears in embedded systems, legacy surveillance hardware, explainable security systems, and educational settings. More broadly, the mathematical tools we use, such as SVD, low-rank approximation, and orthogonal projection, form the foundation of recommendation systems, data compression, bioinformatics, and many other signal processing applications.

A short review of related work and possible approaches to solutions

Among the many face recognition methods developed over the past three decades, several distinct paradigms have emerged. Such methods as Eigenfaces and Fisherfaces treat the entire face as a single vector and rely on linear algebra to find low-dimensional representations. Local feature-based methods such as LBPH capture texture patterns in local regions. Deep learning approaches such as convolutional neural networks (CNNs) learn hierarchical features automatically from large datasets. For this project, we focus primarily on the mathematical comparison between Eigenfaces and Fisherfaces, as both are grounded in linear algebra and offer complementary strengths.

While Eigenfaces uses Principal Component Analysis (PCA) to find directions that maximize the total variance of all face images, regardless of who they belong to, Fisherfaces takes a different approach. Introduced by Belhumeur et al. (1997), Fisherfaces incorporates class labels to explicitly maximize separation between different people.

PCA is unsupervised: it finds the directions in which the data varies the most. However, the directions of largest variance are not necessarily the directions that best separate identities. Fisherfaces solves this by using Linear Discriminant Analysis (LDA), which seeks projection directions that simultaneously bring images of the same person close together and push images of different people far apart.

The mathematical criterion in LDA is the Fisher criterion:

$$J(\mathbf{w}) = \frac{\mathbf{w}^\top S_B \mathbf{w}}{\mathbf{w}^\top S_W \mathbf{w}},$$

where S_B is the between-class scatter matrix and S_W is the within-class scatter matrix. Here, S_B measures how far apart the average faces of different people are, while S_W measures how much images of the same person vary. Maximizing this ratio means finding a direction $\mathbf{w}$ where class means are far apart relative to the spread within each class.

The optimal projection directions are the eigenvectors of

$$S_W^{-1} S_B.$$

These eigenvectors, called Fisherfaces, form a basis optimized for discrimination rather than reconstruction.

Fisherfaces is preferred in security and access-control systems where the environment is relatively controlled but high precision is required between similar-looking individuals, such as employee access to secure facilities, border crossing verification, and forensic face recognition.

A brief explanation of the algorithm chosen and its pros and cons

To solve the face recognition problem, we chose the Eigenfaces method based on Principal Component Analysis (PCA). The core idea is to reduce the dimensionality of the image space. If an image is, for example, 100×100 pixels, then its vector representation has

$$100 \cdot 100 = 10,000$$

dimensions. The Eigenfaces method allows us to reduce this space to a much smaller number of dimensions, called *eigenfaces*, while preserving the most important variations in the data.

The algorithm works as follows:

-
1. All face images are converted to grayscale and resized to the same dimensions.
 2. Each image is flattened into a vector.
 3. The average face is computed and subtracted from all images to center the data.
 4. The covariance matrix of the centered data is computed, followed by its eigenvectors, which form the eigenface basis.
 5. The first k eigenvectors with the largest eigenvalues are selected, since they correspond to the greatest variability.
 6. Each training image is projected into this k -dimensional space, and the same is done for a new test image.
 7. Identification is performed by finding the nearest neighbor using Euclidean distance.

The Eigenfaces method has both advantages and disadvantages.

Advantages:

- simplicity of implementation;
- high computational efficiency after the basis is built;
- the ability to reconstruct the original image from its reduced representation;
- no need for complex model training procedures.

Disadvantages:

- sensitivity to lighting, facial expression, and pose;
- the principal directions of variance are not always the best directions for class separation;
- the parameter k must be chosen carefully;
- if k is too small, important features are lost, while if k is too large, noise is retained.

Despite these limitations, for our dataset, where faces are aligned and lighting conditions are controlled, the Eigenfaces method is stable and avoids the singular matrix problem that arises with Fisherfaces when the number of training samples is small relative to the number of pixels.

The theoretical part behind the algorithm with stress on the LA methods used

First, let us clearly state the problem we are solving. Given a set of face photographs of several people (the training set), we want to determine for any new face image which of these people it belongs to. To do this, we reduce all images to a lower-dimensional space where we can compare them efficiently.

Step 1: Representing images as vectors

Each grayscale image of size $m \times n$ pixels is flattened into a vector

$$\mathbf{x} \in \mathbb{R}^d, \quad d = mn.$$

If we have N training images, we build a matrix

$$X = [\mathbf{x}_1 \quad \mathbf{x}_2 \quad \cdots \quad \mathbf{x}_N] \in \mathbb{R}^{d \times N},$$

where each column is one face vector.

Step 2: Centering the data

We compute the mean face

$$\boldsymbol{\mu} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i.$$

Then we subtract $\boldsymbol{\mu}$ from each column to form the centered matrix

$$A = [\mathbf{x}_1 - \boldsymbol{\mu} \quad \mathbf{x}_2 - \boldsymbol{\mu} \quad \cdots \quad \mathbf{x}_N - \boldsymbol{\mu}].$$

As a result, the centered data have zero mean.

Step 3: The covariance matrix and the problem of high dimensionality

In theory, we need to compute the covariance matrix

$$C = \frac{1}{N} A A^\top,$$

which has size $d \times d$. If d is around 10,000 or more, this matrix is too large to store in memory, let alone to diagonalize directly.

Here we apply an important trick from linear algebra: instead of working with the large

covariance matrix C , we work with the smaller Gram matrix

$$G = A^\top A,$$

which has size $N \times N$, where typically $N \ll d$.

Step 4: Finding the eigenvectors

We want to find the eigenvectors $\mathbf{u}$ of C , that is,

$$C\mathbf{u} = \lambda\mathbf{u}.$$

Substituting

$$C = \frac{1}{N}AA^\top,$$

we get

$$AA^\top\mathbf{u} = N\lambda\mathbf{u}.$$

Now multiply both sides on the left by $A^\top$:

$$A^\top A(A^\top\mathbf{u}) = N\lambda(A^\top\mathbf{u}).$$

Let

$$\mathbf{v} = A^\top\mathbf{u}.$$

Then

$$G\mathbf{v} = N\lambda\mathbf{v},$$

which means that $\mathbf{v}$ is an eigenvector of the small matrix G with eigenvalue $N\lambda$.

Thus, in practice, we first find the eigenvectors $\mathbf{v}_i$ of the matrix G , and then recover the eigenfaces as

$$\mathbf{u}_i = A\mathbf{v}_i.$$

Each $\mathbf{u}_i$ is then normalized to unit length:

$$\mathbf{u}_i \leftarrow \frac{\mathbf{u}_i}{\|\mathbf{u}_i\|}.$$

Step 5: Choosing the number of components k

The eigenvalues λ_i are sorted in descending order. We choose the smallest k such that

$$\frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^N \lambda_i} \geq 0.95.$$

This ensures that at least 95% of the total data variation is retained.

Additionally, in our experiment, we plan to test recognition accuracy for different values of k , for example

$$k \in \{5, 10, 20, 50\},$$

and then select the value that gives the best result.

Step 6: Projection into the eigenface space

Let

$$U_k = [\mathbf{u}_1 \quad \mathbf{u}_2 \quad \cdots \quad \mathbf{u}_k] \in \mathbb{R}^{d \times k}$$

be the matrix whose columns are the first k eigenfaces.

For an arbitrary image $\mathbf{x}$, after centering it by subtracting $\boldsymbol{\mu}$, its projection into the reduced space is

$$\mathbf{p} = U_k^\top (\mathbf{x} - \boldsymbol{\mu}),$$

which is a vector in $\mathbb{R}^k$.

Step 7: Recognition itself

For a test image $\mathbf{x}_{\text{test}}$, we compute its projection

$$\mathbf{p}_{\text{test}} = U_k^\top (\mathbf{x}_{\text{test}} - \boldsymbol{\mu}).$$

Then, among all training projections $\mathbf{p}_j$, we find the one for which the Euclidean distance

$$\|\mathbf{p}_{\text{test}} - \mathbf{p}_j\|_2$$

is minimal. The corresponding training image is identified as the recognized person.

Thus, our approach uses the following linear algebra methods:

- matrix centering;
- the Gram matrix trick to avoid working with a large covariance matrix;
- solving the eigenvalue problem for a small matrix;
- constructing an orthonormal basis;
- dimensionality reduction via orthogonal projection;
- Euclidean distance for comparison in the reduced space.

This is exactly the algorithm implemented in the code linked in our report.

Description of the implementation pipeline

The implementation of the Eigenface Recognition system has already been completed for the initial stage using the standard algorithm based on image preprocessing, matrix construction, mean face calculation, Singular Value Decomposition, projection into the eigenface subspace, and distance-based classification. The main part of the code was written manually according to the mathematical algorithm rather than using ready-made face recognition libraries, which allowed full control over each computational step and a better understanding of the method itself. Initial testing was performed on a standard dataset and confirmed that the recognition pipeline works correctly under controlled conditions. The next planned step is to replace the standard dataset with our own dataset consisting of photographs of real participants. At the moment, we have images of four people, and we plan to expand it by adding four more individuals. After collecting the new data, the full training process will be repeated using our own dataset in order to evaluate the performance of the Eigenfaces method in a more practical and realistic setting.

A link to the working code

<https://github.com/fro-og/Eigenface-Recognition-Manifold-Learning>

Sources

- https://www.youtube.com/watch?v=ofWji_wQBEE
- <https://www.youtube.com/watch?v=yYdYrAKghF4>
- <https://www.youtube.com/watch?v=SsNXg6KpLSU>